

Lecture 34 — Embedded RTOS: FreeRTOS

Jeff Zarnett

2026-09-06

Time, Yes, Time Indeed

The course introduction already introduced the premise of real-time operating systems, and the discussion of real-time scheduling algorithms made it abundantly clear that a typical desktop operating system is not a suitable choice for a real-time system. It is also common for the real-time systems running the most life-and-safety critical things to be embedded; that is, they run on microcontrollers or tiny microprocessors. There's accordingly no capacity on the system to run all the extra parts of the operating system that are not needed.

The scheduling topic also said that a real-time system lives and dies on scheduling. That's true, largely, because the wrong scheduling choices guarantee failure. Meeting deadlines in a real-time system, though, is a weakest-link kind of problem: if any one of the things to be done takes "too long", the deadline is missed.

To get specific about a real-world implementation, we are going to discuss *FreeRTOS*, which is exactly what it says on the label, a free real-time operating system; the project describes itself as a collection of libraries that includes a kernel and the necessary additional functionality [Bar24]. For anything but a system that does exactly one thing at all times, we likely need very little motivation for why we should use an operating system rather than just run everything without one. But if you need some: no need to rewrite things like memory management, scheduling, or power management. You know... just the things we talked about in the course so far... with exceptions.

The FreeRTOS manual describes it as a "library that provides multi-tasking capabilities to what would otherwise be a single-threaded, bare-metal application". This means it does not contain things like device drivers (hardware abstraction) or an I/O framework. We can live without those! They're not essential if the target device is a microcontroller, or at least hardware that is known about in advance. Remember that a big motivator for device drivers is extensibility. In any case, let's look at the things actually present in FreeRTOS.

Tasks. As previously discussed in the real-time scheduling unit, we think about threads as tasks to do in the system. They do not follow the task-hierarchy model of parent and child that we see from UNIX(-like) systems.

The API for creating a new task (analogous to `pthread_create()`) [Bar24]:

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode, /* Function to run */  
    const char * const pcName, /* Name for the task */  
    configSTACK_DEPTH_TYPE usStackDepth, /* Size of the stack */  
    void * pvParameters, /* void* argument to be passed to the function */  
    UBaseType_t uxPriority, /* priority */  
    TaskHandle_t * pxCreatedTask /* Task handle, like pthread_t */  
);
```

It has a real resemblance to creating threads with `pthread_create()`, though it is noteworthy that in this case we explicitly specify both priority and stack size here.

The state model for tasks is simpler than the UNIX model as introduced in ECE 252 and revisited at the start of this course. There are only the states of Running, Ready, Blocked, Suspended, and Deleted [Bar24]. Running, Ready, and Blocked map nicely to what we already know about how threads work. Deleted is used when a task's destruction has happened, while it is being cleaned up. Suspended differs from Blocked in that it is a way of intentionally pausing the task arbitrarily, not when the task is waiting for something. You may think of it as something like an administrative stop; it will resume only when resumption is explicitly ordered, since it is not waiting to get unblocked by an event or interrupt.

It will not be surprising at all to know that FreeRTOS also has an idle task that runs at the lowest possible priority. The motivation here is the same as always: the scheduler implementation is simpler when there will always be some task chosen by the scheduler.

The introduction of the idle task in the scheduling unit said that the idle task can be used as an opportunity to do some tidying up during moments that are not busy. FreeRTOS does exactly that: it uses the idle task to clean up deleted tasks by invoking the function `prvCheckTasksWaitingTermination()` [Bar24].

Memory Management. FreeRTOS does not offer a virtual memory scheme; it instead has five heap allocator approaches and they have to be chosen at compile time [Bar24]:

1. `heap_1`: deterministic memory allocation where memory is allocated by the kernel during setup.
2. `heap_2`: best-fit allocation but deallocation does not perform coalescence. It's considered deprecated now.
3. `heap_3`: uses dynamic allocation with `malloc()` and `free()` which it tries to band-aid to be thread-safe by suspending the scheduler while they run.
4. `heap_4`: first-fit with coalescence.
5. `heap_5`: same as `heap_4` but across non-contiguous regions.

None of those bring to the party anything like virtual memory, so there are no memory protections. That is, any task can see and change the memory of other tasks. It was emphasized repeatedly in the virtual memory topic that there is significant overhead associated with that scheme: looking at page tables and lookaside buffers and all of it do not come for free. So we get faster memory access in FreeRTOS, at the cost of lower safety. Is this an acceptable tradeoff? Certainly for a microcontroller scenario, it's easy to say yes.

This also applies to stack overflowing. The stack size is a design choice in the setup of the system, and if you choose a value that is too small then tasks will overflow their buffers. Buffer overflows in these scenarios result in corruption of other parts of memory that may be hard to notice until serious damage is done.

A partial solution to this is another form of canary. Much earlier on, we talked about the use of a canary value to stop a malicious actor from redirecting control of execution to the attacker's desired location. If the end of the stack is marked with a (different) canary value; if the stack overflows its normal bounds, that value will have been overwritten. The kernel can check, on a context switch, if this value has been overwritten and if so, identify the problem. There's a handling function the kernel will call and you can decide to terminate the task or do something else. But this is after-the-fact; memory has already been corrupted.

FreeRTOS does offer the stack canary option and uses the magic value `0xa5a5a5a5`, which you may observe is a fixed value and that works because this is a debugging tool rather than a security mitigation; FreeRTOS also can calculate whether the saved stack pointer is within the designated stack area for the task [Bar24].

Scheduling. The scheduling unit introduced several real-time scheduling algorithms (earliest deadline first, least slack first, rate monotonic, and the aperiodic server approaches), and yet FreeRTOS uses something else: Fixed Priority Preemptive Scheduling; which can be with or without time slicing [Bar24]. This is very similar to the Highest Priority, Period algorithm we discussed previously, optionally with time slicing between tasks at the same level. The number of priorities is capped at word size, and rather like the $O(1)$ scheduler, this permits finding the first ready task in constant time. The architecture may permit a count-leading-zeros instruction to find the first task rapidly and in a fixed amount of time.

Okay, it does also offer co-operative scheduling, which you might recall was in Mac OS 9 and earlier. This is a valid option in a RTOS scenario where we have control over all threads and tasks and be sure they play nicely. It does not work in a general purpose sense where tasks can be defined by anyone and do not necessarily co-ordinate well about how they will get along.

You might also question why this scheme is used when Earliest Deadline First is provably optimal. The most practical reason is that FreeRTOS does not have any formal way to put that information into the scheduling routine, though it's not impossible¹. That is a partial answer to why it's not one of the available choices, but is not a clear answer to why the system isn't designed this way in the first place.

To quote a research paper on why RTOS systems don't choose this in general: "This is done for reasons of simplicity and lower overheads: in particular, the number of priority levels is often limited allowing a bit-map representation of the ready queue to be used, resulting in $O(1)$ queuing and de-queuing operations" [Dav14]. Or in other words, while we know that this algorithm is not provably optimal, that just means we need a little padding of capacity to be sure that we'll always succeed... remember, knowing exactly how long it takes to find the next task is the most important goal.

Inter-Process Communication. ECE 252 covered different methods for inter-process communication. FreeRTOS favours queues (i.e., message passing) as the mechanism for this. It uses copy semantics: the message is copied into the queue, byte for byte [Bar24]. This may feel unfamiliar and wasteful of effort vs. queuing by reference; why make the copy? The docs give many reasons for this, but some salient ones are things like (1) data can be sent directly from the stack into the queue without needing to be dynamically allocated first; (2) the sending task can re-use the memory space immediately instead of waiting for it to be consumed; and (3) there are no headaches around who is responsible for deallocation.

An analogy that ECE 252 used here was to think of messages like physical mail letters that are placed into the mailbox. That is a reasonable mental model for the queuing by reference approach. But who sends physical mail these days? Queuing by copy is much more like e-mail. When I send an e-mail, I send a copy of the content to my e-mail server, which will then send the copy to your e-mail server (if it's a different one) and when the e-mail is delivered to your client, you get a copy of it. After I've sent the e-mail, I can delete the content on my side or re-use that space for something else.

When there are multiple tasks blocked waiting for a given queue, unblocking follows scheduler rules; the priority of the task is most important, and if there is a tie then the task waiting the longest is unblocked first [Bar24].

In a world of no memory protection, of course, all memory is shared memory. This does potentially permit avoiding the whole conversation around IPC and just telling everyone to use mutual exclusion constructs. That is not illegal, of course, but I would recommend a more structured mechanism for inter-process communication because being explicit about it makes it much easier to audit if it is correct.

Mutual Exclusion. FreeRTOS provides a few mechanisms of concurrency control. The first one is disabling interrupts by designating the start and end of a critical section as in this code example [Bar24]:

```
void vPrintString( const char *pcString )
{
    /* Write the string to stdout, using a critical section as a crude method of mutual exclusion. */
    taskENTER_CRITICAL();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    taskEXIT_CRITICAL();
}
```

But you remember why this is a bad idea, no doubt: it's crude and disables interrupts that might be rather important. And it does not solve the problem in a multicore system. There's a related approach discussed of suspending and resuming the scheduler, which gets around the problem of turning off interrupts, but it's still a bad plan. There are of course the better options: mutex and semaphore constructs as you would expect, with the mutex, binary semaphore, and general (counting) semaphore options.

Interestingly, in FreeRTOS the mutex has priority inheritance but the binary semaphore does not [Bar24]. The

¹<https://forums.freertos.org/t/scheduler-algorithms/8551/3>

manual emphasizes that priority inheritance does not truly fix priority inversion, but it does lessen the impact of the inversion.

Consistent with the behaviour of the scheduling algorithm, when a thread unlocks a mutex or posts on a semaphore, if tasks are waiting, unblocking takes place according to task priority.

Power Management. Microcontrollers and tiny embedded systems also frequently have tiny batteries, unless of course they run when plugged in. But a lot of systems really do have to think about power management. The OS typically has a timer tick, but if there is nothing to do right now, disabling that is actually a way to save power. Waking up to handle the tick every x milliseconds is a waste of effort and energy.

FreeRTOS offers the idea of tickless idle: just wait in idle mode without waking up for ticks periodically until the next scheduled event. The manual describes how long to sleep as “the total number of tick periods before a task is due to be moved into the Ready state. The parameter value is therefore the time the microcontroller can safely remain in a deep sleep state, with the tick interrupt suppressed.” So if we know that the next task will become ready after x time, what if we just sleep until x time has elapsed and not wake up in between?

Of course, if there is an interrupt, that will wake up the system. Ah, about interrupts...

Interrupt Management. Interrupt handling, as expected, is a critical part of most embedded RTOS implementations. How quickly the event is handled is critical to the idea of meeting the deadline. There’s nothing new to say about where interrupts come from or what one looks like; only how they are handled.

Interrupts trigger interrupt service routines (ISRs) which process the event in some way. ISRs take precedence over normal task execution, and there is always pressure to keep the handler execution time as short as possible, ideally by using the interrupt handling to set up a task to handle the event later. Setting up the task to run later is known as *deferred interrupt processing* and is generally recommended, because while an ISR is running it not only prevents tasks from running, it also may disrupt other interrupts. Interrupts can fire at any time; the higher priority one can interrupt the currently-executing one, and if the currently-executing one is high-priority and long-running, it’s holding up the execution of everything else.

Remember we talked about the idea that invoking a function in a signal handler requires checking that the function has a designation *async-signal-safe*? The FreeRTOS API is divided into functions that either do or do not contain the words `FromISR` in their name, making it extremely obvious whether or not it is okay to call [Bar24]. The book identifies that as a design decision, it’s easier to just have two different functions – ISR and non-ISR variants of the same function – than it is to try to figure out inside that function how we got there. Could it be a boolean flag in the arguments? Maybe, but someone whose technical opinions I respect also suggested the idea that a boolean parameter in a function is actually an anti-pattern and there should be two functions instead.

Here, two examples for sending things into the back of a queue, showing the difference [Bar24]:

```
BaseType_t xQueueSendToBack(
    QueueHandle_t xQueue,
    const void * pvItemToQueue,
    TickType_t xTicksToWait /* How long to block when trying to add if the queue is full */
);

BaseType_t xQueueSendToBackFromISR(
    QueueHandle_t xQueue,
    const void * pvItemToQueue,
    BaseType_t * pxHigherPriorityTaskWoken /* Set to true if enqueueing has unblocked something higher priority */
);
```

The final parameter is different in these two cases! In the non-ISR version, there is a maximum time to wait if the queue is full. This is a part of how, of course, to put a maximum execution time on a task (even if it is unsuccessful in adding the item to the queue). In the ISR version, this parameter is set if adding the queue woke up something else higher in priority. That means the newly-unblocked task should resume after the interrupt has completed

rather than whatever task was executing before the ISR. The ISR needs to use this value at the end of its function to make sure execution goes to the right place.

Timers. Unusually, software timers in FreeRTOS are tasks rather than interrupts. This means the timer tick is really the only interrupt related to timing. At each timer tick, the OS will then figure out whether the timer task should become ready to run and change its state accordingly. Then, it will get scheduled based on the scheduling algorithm as expected. All the timers share one context (which may be unexpected).

As previously discussed getting the timing exactly right on events is difficult. FreeRTOS makes a provision for the difference in time between submitting a command and its processing: “Commands sent to the timer command queue contain a time stamp. The time stamp is used to account for any time that passes between a command being sent by an application task, and the same command being processed by the daemon task. For example, if a ‘start a timer’ command is sent to start a timer that has a period of 10 ticks, the time stamp is used to ensure the timer being started expires 10 ticks after the command was sent, not 10 ticks after the command was processed by the daemon task” [Bar24].

One Operating System, To Go, Please. All of the kernel implementation of FreeRTOS is located in just seven C source files. That’s pretty minimal, and it’s remarkably effective for this rather small footprint. Yes, the memory management strategies are outside of that.

A late-breaking challenger appears: the Linux Kernel added the mode `PREEMPT_RT` in 2024 (after like 20 years of work to get it accepted) that lets Linux be used for hard-real-time tasks [Cor24]. Why not use that? For a bigger system, sure! But Linux itself is large and complex and expects things like memory management units. That’s too much overhead for a microcontroller scenario. FreeRTOS fits nicely in a smaller space.

Is this enough for life- and safety-critical systems? We might perhaps like more assurance. A variant of FreeRTOS, called SAFERTOS, follows the same functional model but has a different implementation, with no open-source code, that is externally verified by TÜV SÜD (Technischer Überwachungsverein, a German-based technical inspection agency) [WIT24]. This is what you would want to use if you really intended to build such a system.

It costs money to get a certified OS, but paying for assurance is part of building a reliable system. Just as meeting deadlines is a weakest-link problem, every component in a life- and safety critical system needs assurance to say that the whole system will perform as expected.

References

- [Bar24] Richard Barry. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Amazon Web Services, 1.1.0 edition, 2024. Referencing FreeRTOS kernel V10.6.2. Available at <https://github.com/FreeRTOS/FreeRTOS-Kernel-Book>.
- [Cor24] Jonathan Corbet. The realtime preemption end game — for real this time. LWN.net, September 2024. <https://lwn.net/Articles/989212/>.
- [Dav14] Robert I. Davis. A review of fixed priority and EDF scheduling for hard real-time uniprocessor systems. *ACM SIGBED Review*, 11(1):8–19, 2014. URL: <https://www-users.york.ac.uk/~rd17/papers/SingleProcessorSchedulingReview.pdf>, doi:10.1145/2597457.2597458.
- [WIT24] WITTENSTEIN high integrity systems. SAFERTOS, OPENRTOS and FreeRTOS services. Company brochure, version 8.6, 2024. https://www.highintegritysystems.com/wp-content/uploads/2024/05/brochure_v8.6.pdf.